2024/05/23 Progress Report

Yuri Guimaraes

May 23, 2024

Outline

1. Summary
2. Hardware-Software Architecture
3. GStreamer
4. Source Code Analysis
5. Conclusion

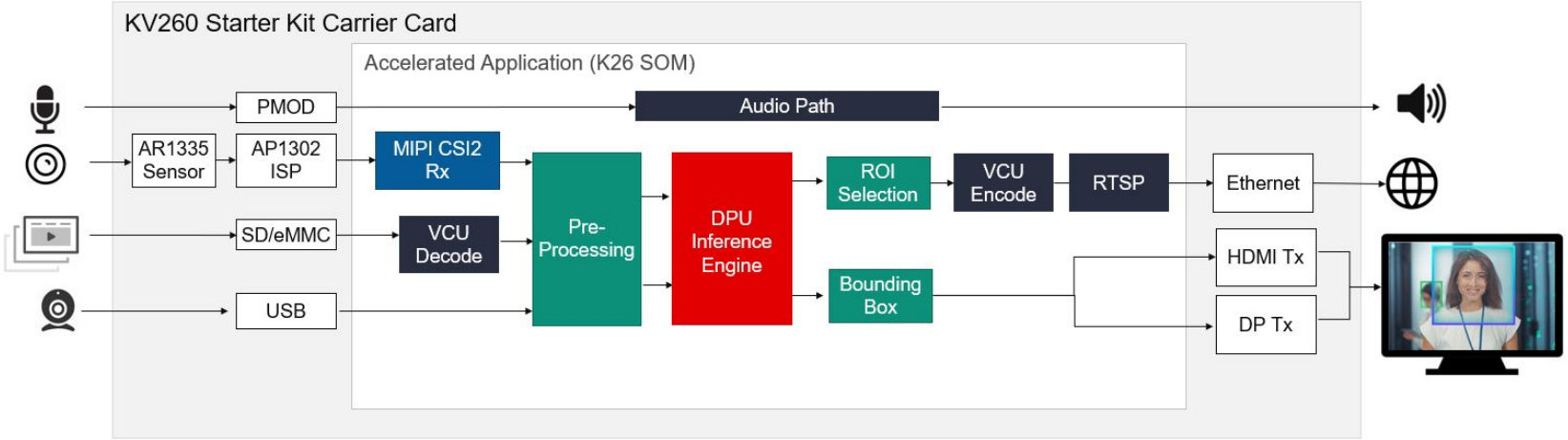
Summary

Task List

1. ~~Browse `smartcam` module source code.~~
2. ~~Compile `smartcam` code.~~
3. ~~Modify `smartcam` code to use other kernels.~~
4. Write image compression $\rightarrow$ storage prototype pipeline.

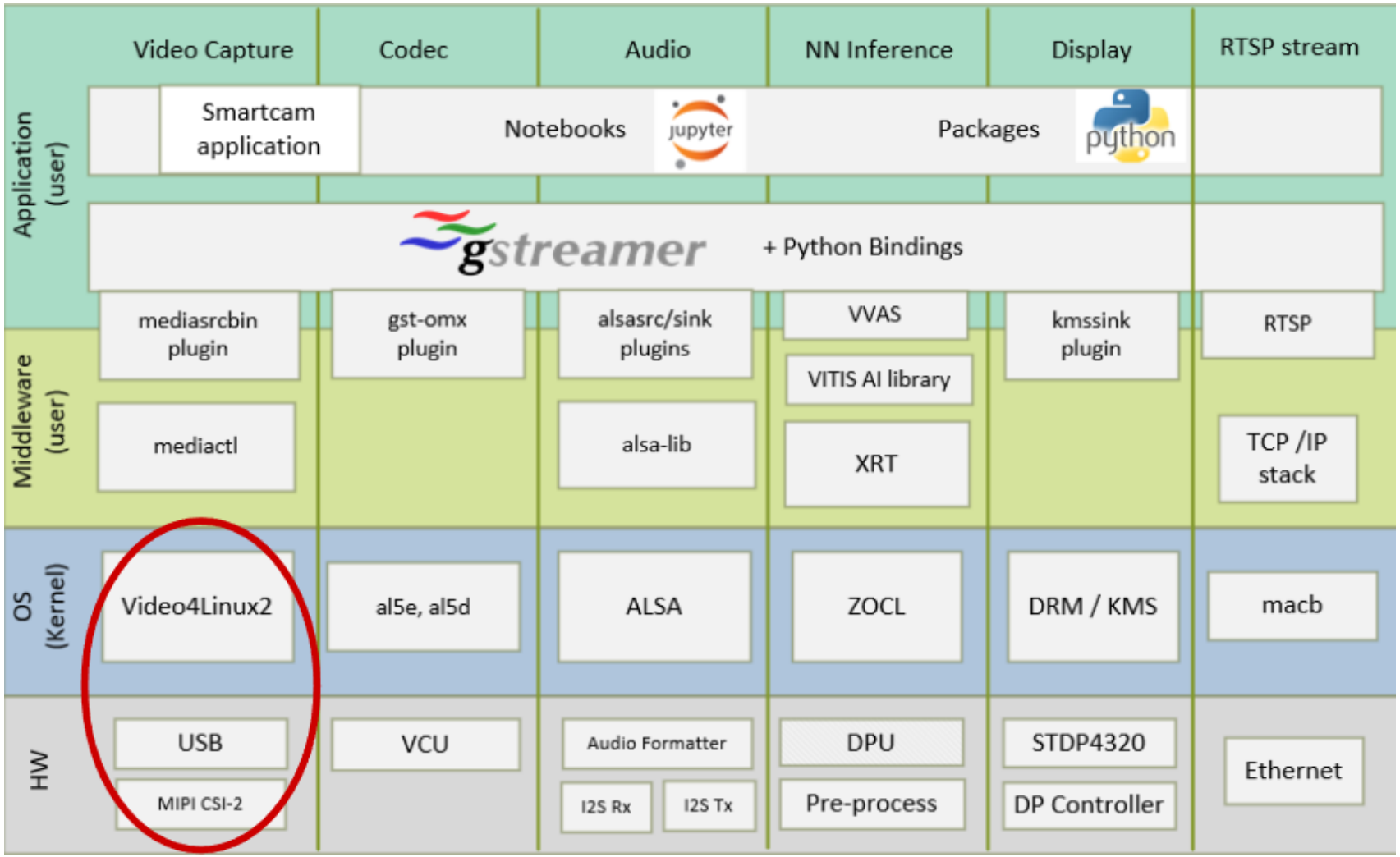
Recap (1 / 2)

Xilinx's Smart Camera application allows for a
`MIPI capture` → `PL processing` → `RTSP/DP output` pipeline.



Recap (2 / 2)

This is done via a GStreamer-Video4Linux2 driver.



Objectives

Runtime

- What is v4l2's role in acquiring the image from DMA?
- How does GStreamer use v4l2?
- How does an acceleration kernel run using v4l2/GStreamer?

Building

- What does the final software depend on?
- What does the developer need to have ready in order to build the application?
  - Dependencies?

MIPI-CSI2 Capture (Hardware)

- MIPI source outputs to `S_AXI_HP0_FPD`.
- `NV12` format.

NV12	Planar 8-bit YUV 4:2:0
YUV420	Semi-planar 8-bit YUV 4:2:0

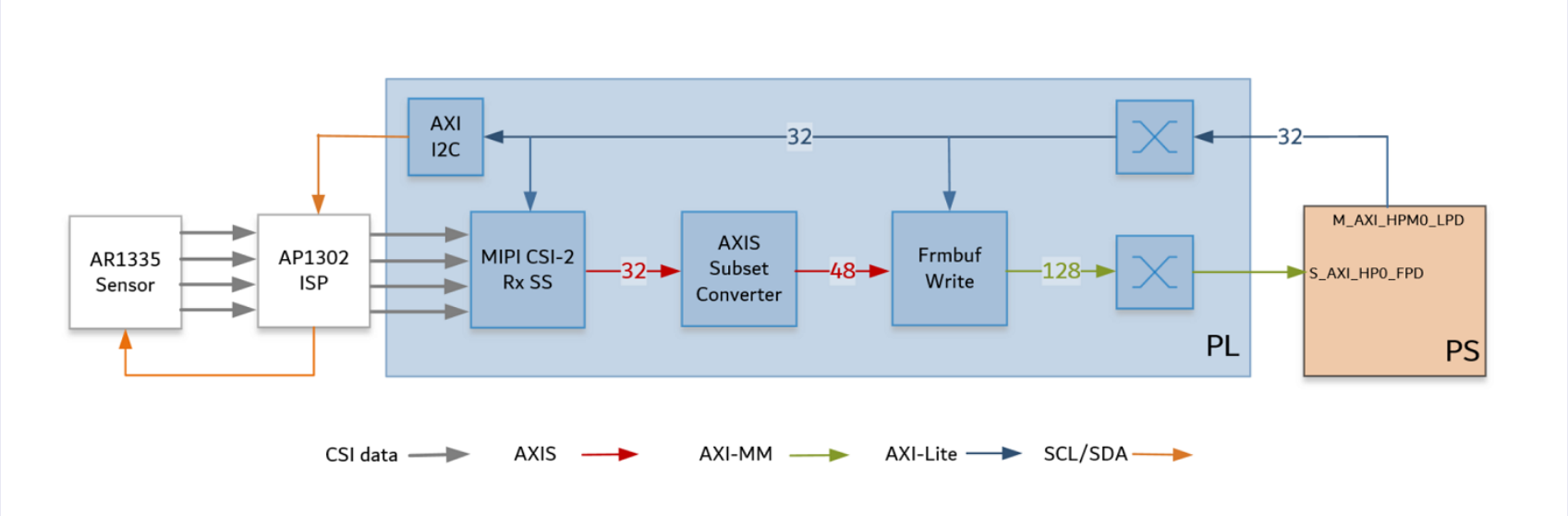


Figure: MIPI Capture Pipeline

MIPI-CSI2 Capture (Software) (1 / 2)

- Source accessible via v4l2
  - → /dev/media*
  - → /dev/video*
  - ...
  - → v4l2src
  - → mediasrc

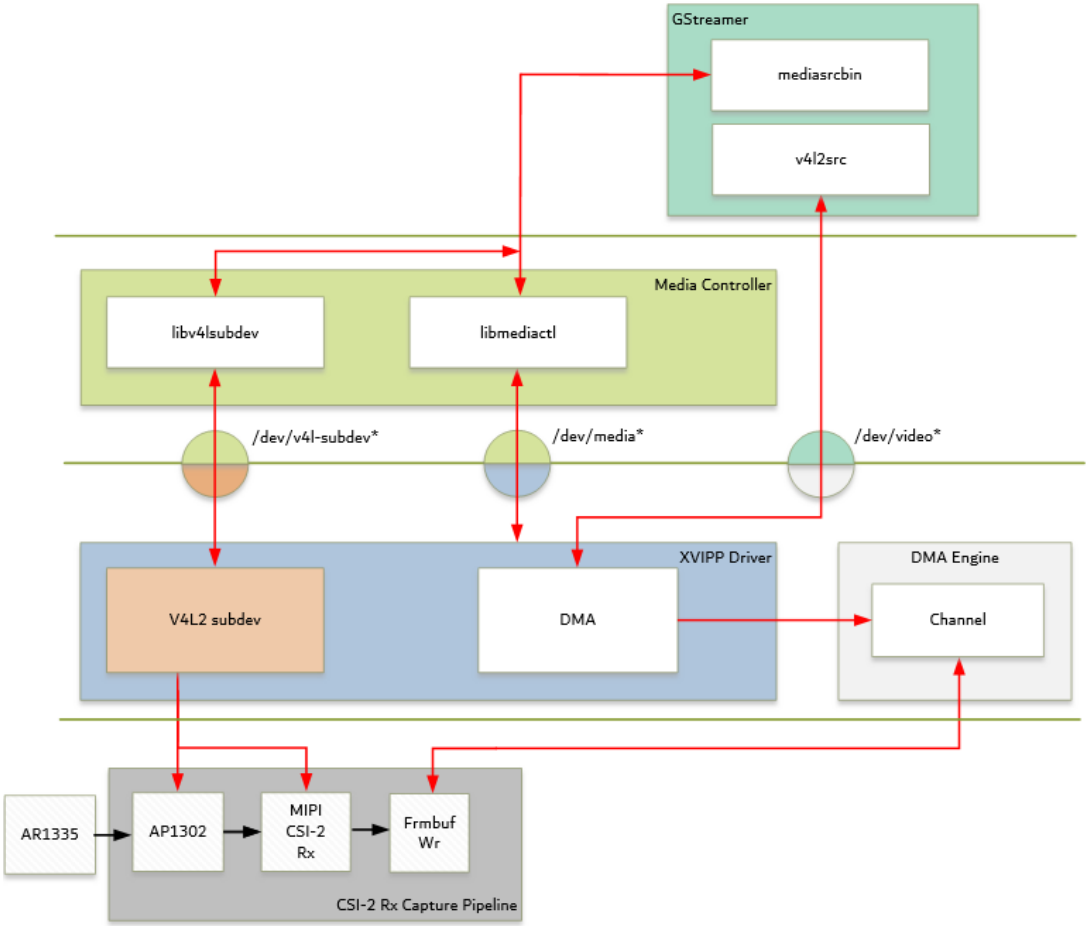


Figure: Video Capture Diagram

MIPI-CSI2 Capture (Software) (2 / 2)

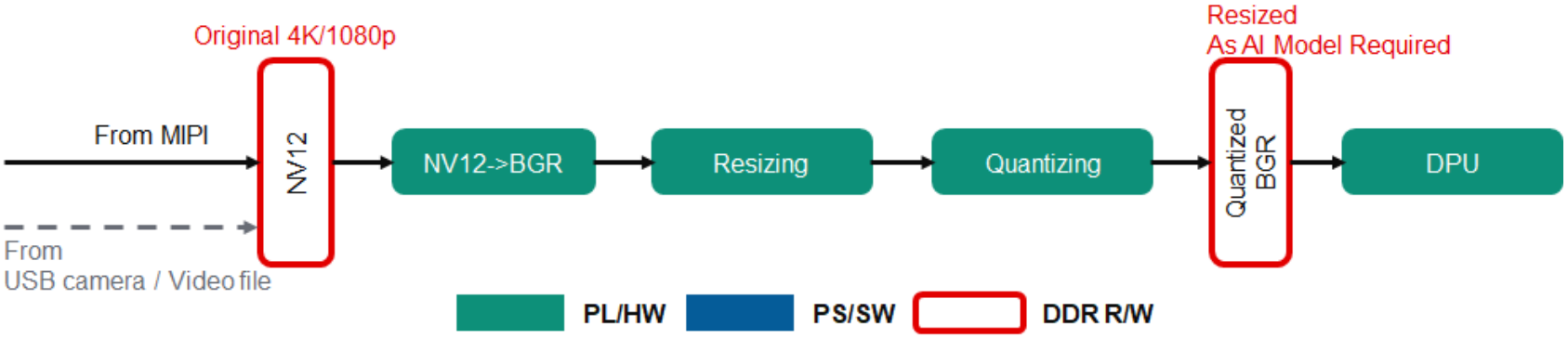


Figure: GStreamer Pipeline

Acceleration

Acceleration on the new stack is based on HLS kernels. For vision, there is `VVAS`. The Smart Camera application uses `VVAS` with `GStreamer`.

VVAS

Vitis Video Analytics SDK is Xilinx's software stack for building video processing applications.

Plugin Structure

Use GStreamer plugins for creating computer vision pipelines.

GStreamer

GStreamer is a pipeline-based multimedia framework linking various media processing systems to create workflows.

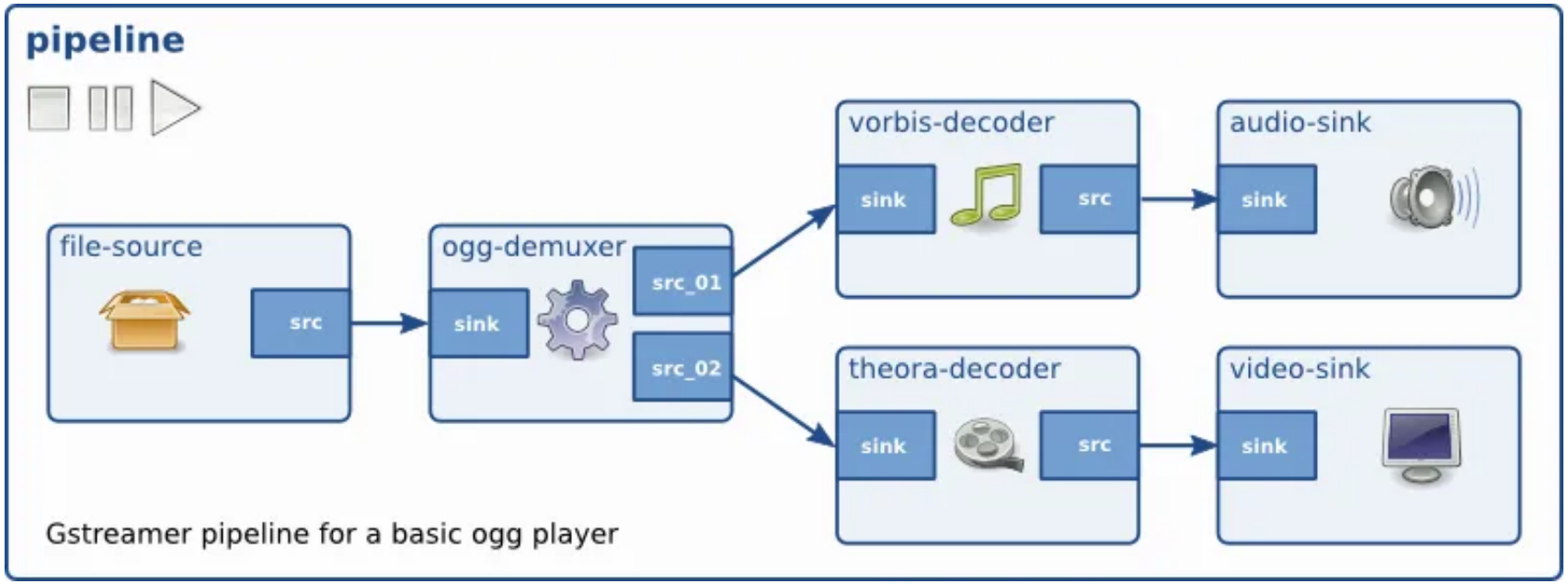


Figure: Example of a GStreamer pipeline.

GStreamer Basics

Elements

Objects derived from `GstElement`. Examples:

- `source` element: provides data
- `filter` element: acts on incoming data

Pads

Objects derived from `GstPad`. They are the "ports" that intermediate data between elements.

- *Source* pad: data flows outward
- *Sink* pad: data flows inward

Buffers

- `GstMiniObject`, `GstBuffer`, `GstEvent`...

GStreamer Plugins

- Enables plug-and-play functionality with GStreamer.
- Elements must be wrapped into a plugin before being used.

What is a plugin?

Essentially, it is a block of loadable code (a shared object/dynamically linked library).

What is the structure of a plugin?

`https://gitlab.freedesktop.org/gstreamer/gst-template.git`

- The above serves as good reference.
- The compiled code generates a `libgstplugin.so`, which can be loaded by `gst-launch-1.0` by simply specifying the path in `GST_PLUGIN_PATH`.

VVAS

- Collection, as well as an API for creating acceleration kernels for usage with GStreamer.

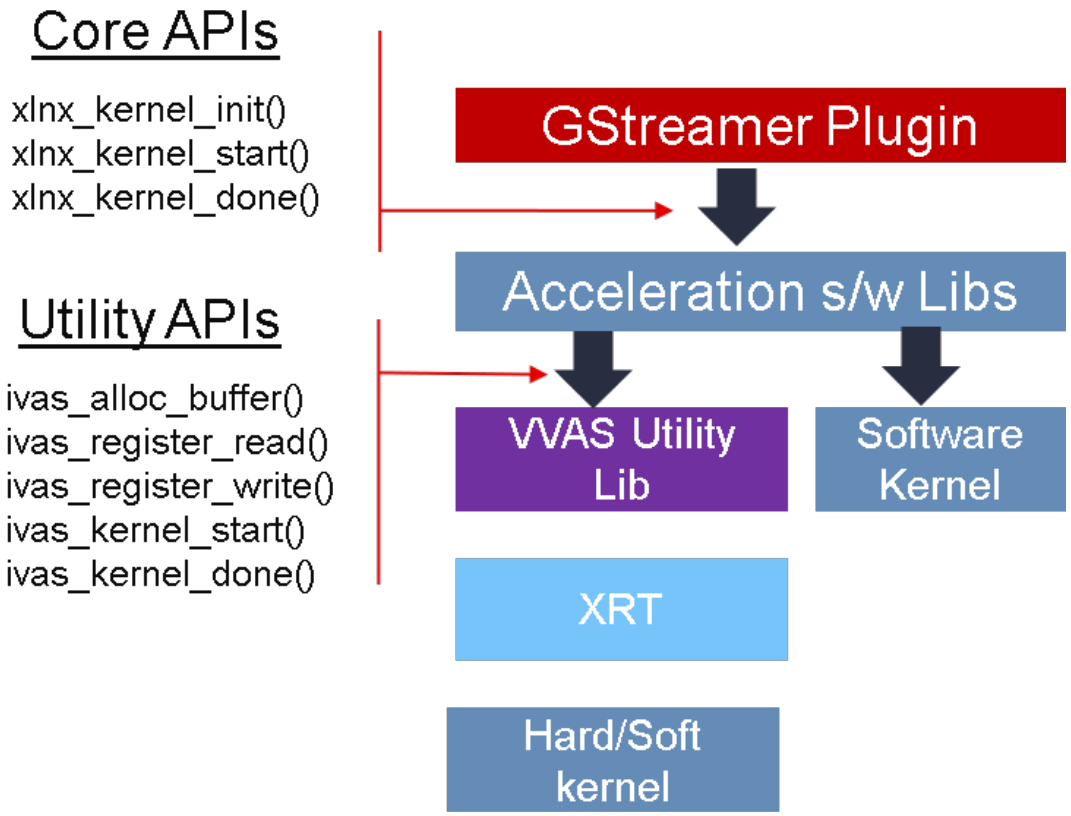


Figure: How VVAS relates to GStreamer

VVAS Usage in GStreamer (1 / 2)

API usage:

1. Plugin Initialization (`xlnx_kernel_init`)
2. Kernel Start (`xlnx_kernel_start`)
3. Waiting for the kernel to finish (`vvas_kernel_done`)
4. Denitilizating the plugin (`xlnx_kernel_deinit`)

VVAS Usage in GStreamer (2 / 2)

`vvas_xfilter`:

JSON config

Contains information regarding:

- `xclbin` location
- `vvas-library-repo`
- `element-mode` (passthrough, inplace, transform)

`xclbin`

Used to program the FPGA.

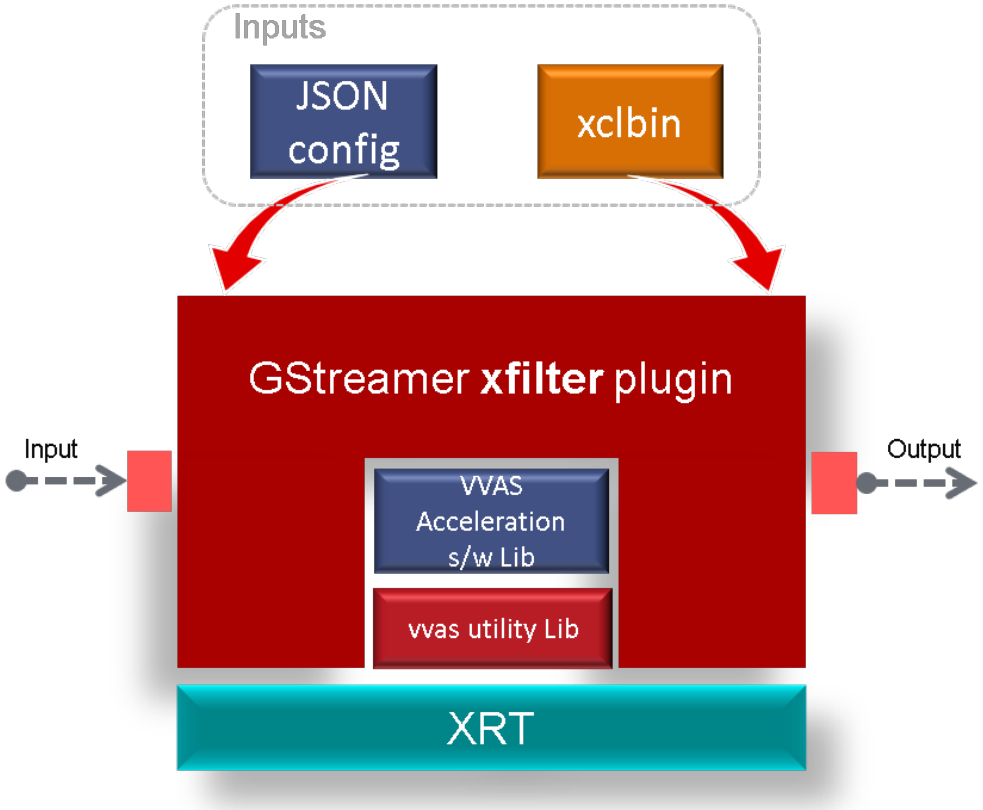
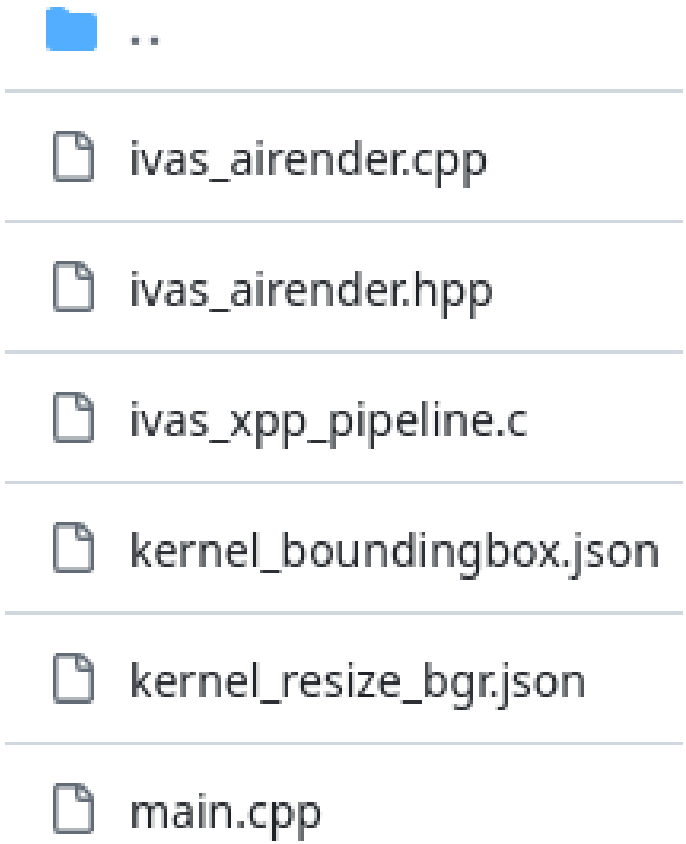


Figure: `xfilter` usage.

Smart Camera — Folder Structure

- `smartcam`: Contains the base smartcam application. It is built and used with Docker.



`smartcam/src`

`ivas_airender`: aadasda
`ivas_xpp_pipeline`: aadasda
`main`: Contains the GStreamer pipelines that run everything.

VVAS Usage in smartcam

`vvas_xfilter`: Plugin to act directly on data with the specified kernel.

```
if (!nodet) {
    sprintf(pip + strlen(pip), " ! tee name=t \
            ! queue ! ivas_xmultisrc kconfig=\"%s/preprocess.json\" \
            ! queue ! ivas_xfilter kernels-config=\"%s/aiinference.json\" \
            ! ima.sink_master \
            ivas_xmetaaffixer name=ima ima.src_master ! fakesink \
            t. \
            ! queue max-size-buffers=1 leaky=%d ! ima.sink_slave_0 ima.src_s
            confdir.c_str(),
            confdir.c_str(),
            filename? 0 : 2, confdir.c_str());
}
```

VVAS Example — Edge Filter with `filter2d` (1 / 3)

- `filter2d` source code

Makefile

```
filter2d_xo = filter2d_pl/kernel/filter2d_pl_accel/filter2d_pl_accel.xo


$(BINARY_CONTAINER): $(filter2d_xo)
        $(VPP) $(VPP_LINK_FLAGS) -o $@ $(+)
        $(XBU) $(XBU_FLAGS) --input $(BINARY_CONTAINER) --output tmp.xclbin
        -@$(MV) tmp.xclbin $(BINARY_CONTAINER)
        -@$(RM) .Xil

$(filter2d_xo):
        $(MAKE) -C filter2d_pl/kernel


        $(MAKE) -C filter2d_pl/kernel clean
```

VVAS Example — Edge Filter with `filter2d` (2 / 3)

- `filter2d` source code

PS Connectivity

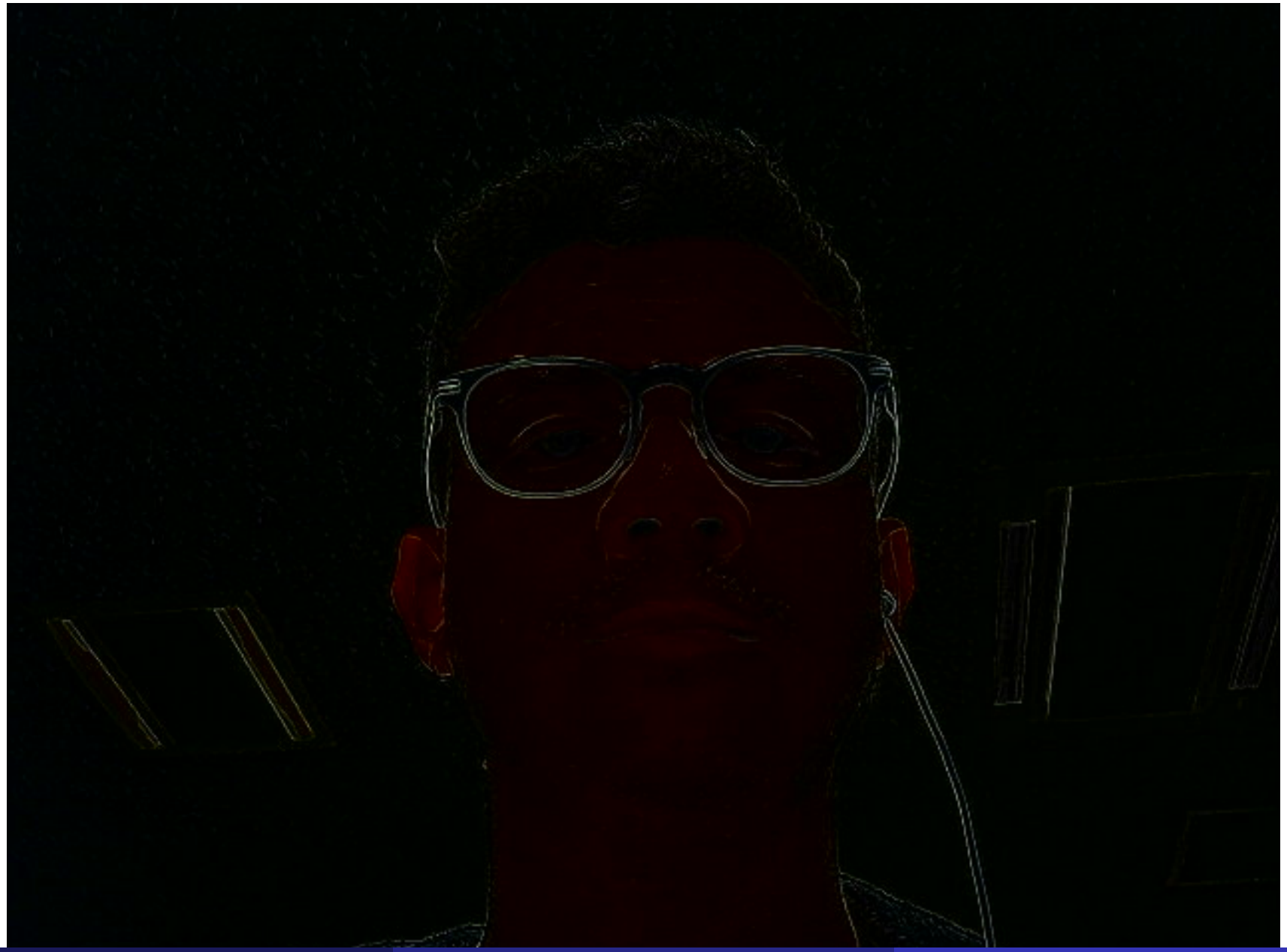
```
[clock]
#freqHz=300000000:DPUCZDX8G_1.aclk
#freqHz=600000000:DPUCZDX8G_1.ap_clk_2
#freqHz=300000000:pp_pipeline_accel_1.ap_clk

freqHz=200000000:filter2d_pl_accel_0.ap_clk

[connectivity]
#sp=DPUCZDX8G_1.M_AXI_GP0:HP1
#sp=DPUCZDX8G_1.M_AXI_HP0:HP1
#sp=DPUCZDX8G_1.M_AXI_HP2:HPC1
#sp=pp_pipeline_accel_1.m_axi_gmem1:HP3
#sp=pp_pipeline_accel_1.m_axi_gmem2:HP3
#sp=pp_pipeline_accel_1.m_axi_gmem3:HP3
#sp=pp_pipeline_accel_1.m_axi_gmem4:HP3

nk=filter2d_pl_accel:1:filter2d_pl_accel_0
sp=filter2d_pl_accel_0.frm_in:HP1
sp=filter2d_pl_accel_0.frm_out:HP1
sp=filter2d_pl_accel_0.kernel:HP1
```

VVAS Example — Edge Filter with `filter2d` (3 / 3)

Launching GStreamer with filter2d kernel

```
gst-launch-1.0 v4l2src \
  device=/dev/video0 io-mode=mmap !\
  "video/x-raw, width=640, height=480" !\
  videoconvert !\
  "video/x-raw, width=640, height=480,
   format=YUY2, framerate=30/1" !\
  vvas_xfilter kernels-config=/tmp/kernel_xfilter2d_pl.json\
  dynamic-config='{ "filter_preset" : "edge" }' !\
  perf ! kmssink plane-id=39 fullscreen-overlay=true -v
```

VVAS Example — Results of the Edge Filter



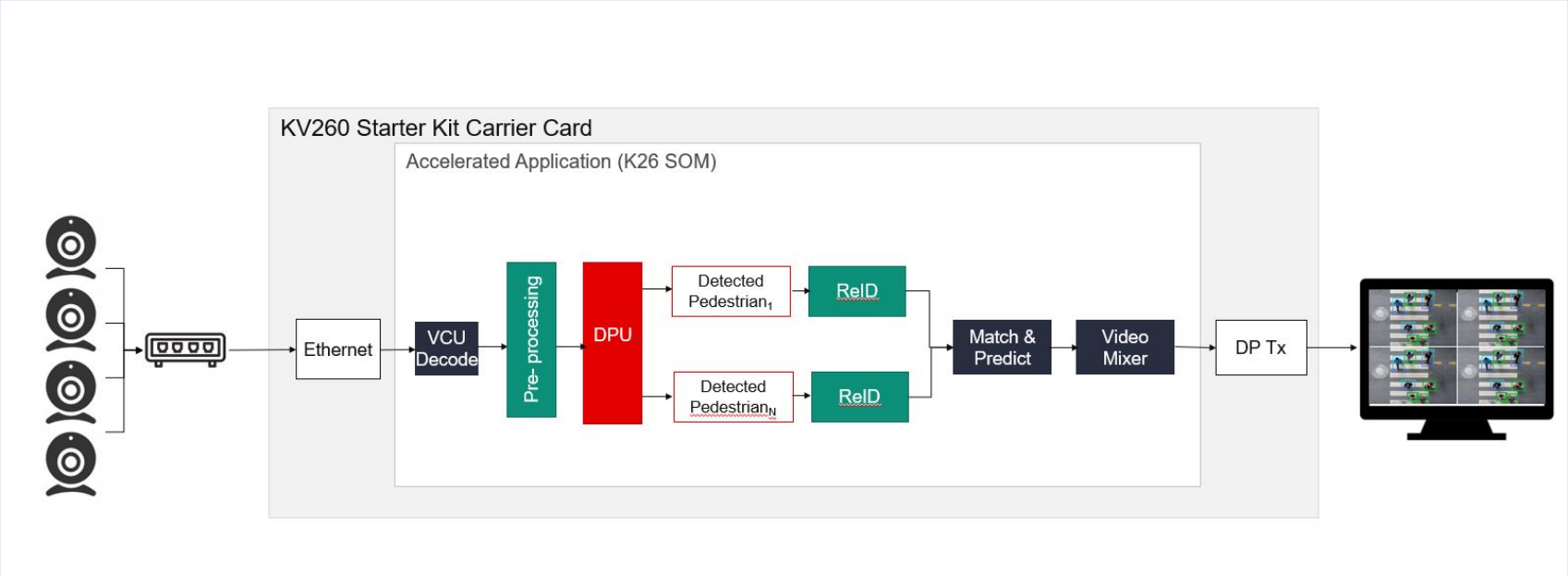
Conclusion

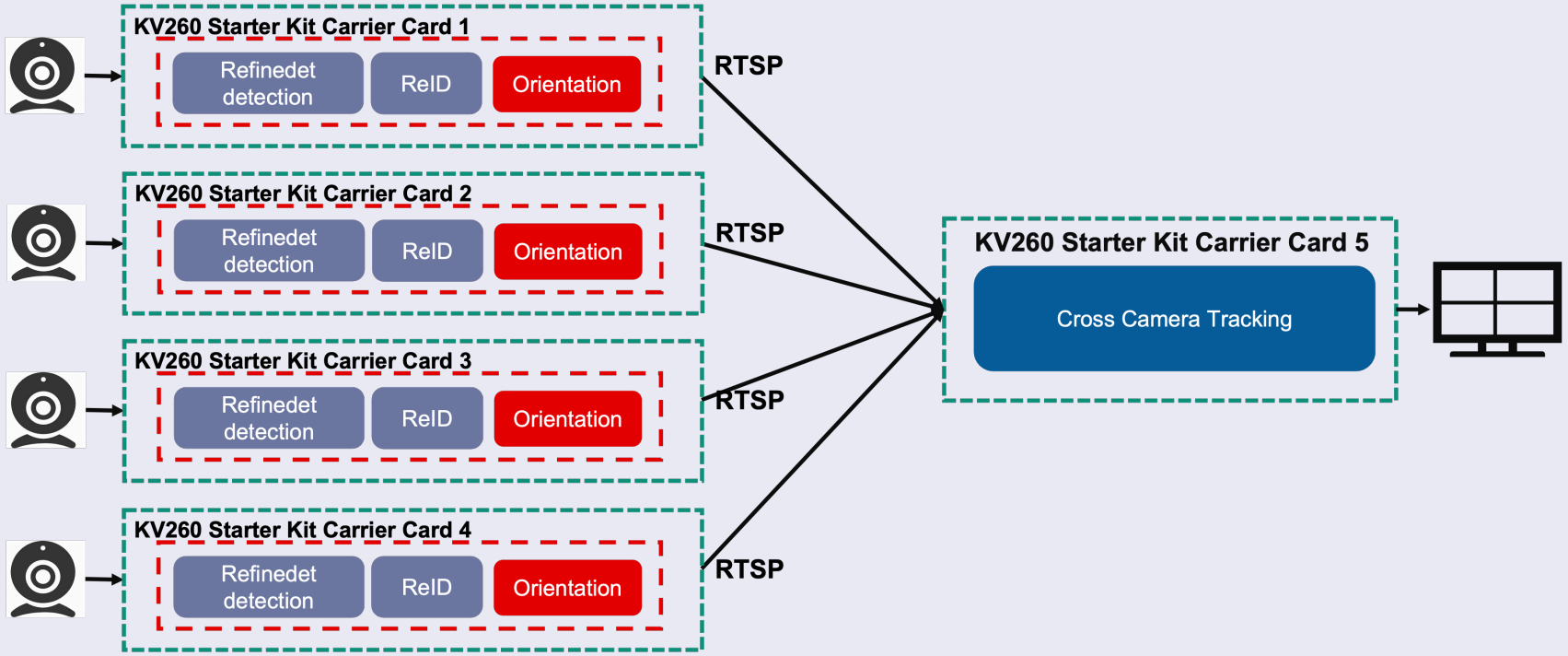
- Got smartcamera to work on Docker inside the KV260
  - It also worked with a modified kernel
- Next: Write image compression $\rightarrow$ storage prototype pipeline?
- I'm also working on making the build process for these packages simple.

Appendix

AIBox-ReID

Xilinx demo for 4k IP cameras as inputs on the KV260.



AI Box Distributed ReID

Distributed demo.